Parse2Vec: Embeddings Deeper than Surface Forms

MICHAEL CONNOR

Objective: The minimum transformer performance can move forward by incorporating the progress of the last decade. **Background:** Current efforts begin with surface forms and build meanings for tagging, parsing, identifying semantic relationships, etc. But a parser and outside resources can also provide these morphological, syntactic, and semantic features. **Method:** Instead of inputting tokens of surface forms into a transformer, we could start with token vectors whose summed embeddings represent the deep features of the words. This is an empirical study on how inputs with token vectors perform on masked word prediction.

JAIR Associate Editor: Insert JAIR AE Name

JAIR Reference Format:
Michael Connor. 2026. Parse2Vec: Embeddings Deeper than Surface Forms. *Journal of Artificial Intelligence Research* 4, Article 6 (April 2026), 6 pages. DOI: 10.1613/jair.1.xxxxx

1 Background

The latent information in a transformer model and in word embeddings covers a diverse field, even of just what can be input or output. The impetus for this work is (Mikolov, Sutskever, et al. 2013), (Mikolov, Yih, et al. 2013), and (Mikolov, Chen, et al. 2013), who catalogued syntactic and semantic word relations, and showed how words are composible through their relationships. In (Bian et al. 2022), surface form words, root words, suffixes, and parts of speech form various feature vectors for word embedding reasoning tasks. In (Liu and Hou 2024), surface forms and dependencies combine for translations. In (Jabbar 2024), affixes and root words are derived with byte pair encoding plus a morphological segmentation table, in order to produce better sub word tokenization. In (Bolukbasi et al. 2016), gendering in words introduces bias in word embeddings. In (Egea Gómez et al. 2021), spaCy output including dependencies create a gloss representation for translation. In (Stylianou and Vlahavas 2020), entities and coreferences combine within the transformer during training. In (Vaswani et al. 2017), related absolute positions were found to be equivalent but preferred to learned absolute positions. In (Shaw et al. 2018), relative positions outperform absolute positions in some tests. And the Transformer of (Vaswani et al. 2017) uses surface forms, but can easily accomodate multimodal input.

All of these seem unrelated, but all of these come from latent information within the transformer model and all of these can be output by preprocessing with a parser and outside resources[1]. Given the conclusions of past work that transformer models contain these relationships from surfarce form text, a transformer model with these relationships as input should have comparable results.

[1]https://github.com/maconnor34/parse2vec

Author's Contact Information: Michael Connor, maconnor34@gmail.com.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2026 Copyright held by the owner/author(s).
DOI: 10.1613/jair.1.xxxxx

2 Method

The training corpus goes through the spaCy (Honnibal et al. 2020) parser and combines with outside resources. Among other parsers tested, spaCy has a free and publicly available version with ample documentation. The outside resources are original but simple csv tables.

Each input word is represented by a vector of features rather than by the surface form alone. The transformer model is multimodal and uses early fusion. The output is surface form tokens for training and masked word prediction.

A transformer using surface form input and with surface form output marks the baseline for comparision. Masked word prediction relates directly to the model training and reflects a basic function of a transformer model. The metrics for the experiment are accuracy and perplexity of predicted words, excluding words out of the vocabulary with all numbers being out of vocabulary. The training and testing use different corpora that share a theme of news, to put more distance between training and testing.

Configuration

The tests utilize the AG News corpus (Zhang et al. 2015) for training and Newsgroups20 corpus (Lang 1995) for testing. All data comes from a MSI GF75 Thin laptop with a Nvidia RTX 3090. The volume of data in training is 2.5M words with 19K unique words and in testing it is 250K words.

The transformer architecture settings are: sequence length = 16, attentation heads = 4, layers = 3, hidden dimension = 256, feed forward dimension = 1024, dropout = 0.1, optimizer = Adam with no other settings. These settings fit the hardware.

Preprocessing:

The publically available spaCy produces *token* objects containing *text*, *tag* which are Brown tags (Francis Kučera and W. Nelson Francis 1979), *pos*, *lemma*, *dep* which are Standford tags (Marneffe and Manning 2008), *morph*, and *ent_type* from en_core_web_lg and *coref_clusters* from en_coreference_web_trf. Based on these, head dependency tags and more position tokens are created. To the spaCy output, outside resources add root words, suffixes, sub-suffixes, and gender features from csv tables.

Features like the elements of spaCy's *token* appear multiple times in previous work, often in pairings like the surface form and dependency couples of (Egea Gómez et al. 2021) and (Liu and Hou 2024). Relating to dependencies, a head dependency tag is a copy of the dependency tag attached to the head of the compliment or modifier and marked as head. For example in "brown fox", *amod* attaches to "brown" and *head_amod* attaches to "fox". The data from *ent_type* is included for its syntactic and semantic categories. Incorporating coreferences happens here before the transformer and copies the practice of (Stylianou and Vlahavas 2020) in replacing the pronouns with the entities. Root words, suffixes and sub-suffixes are not sub word tokens and are not defined by byte pair encoding like in (Jabbar 2024). They are copied from a morphological segmentation table with major suffix combinations. Gender features come from a list of common gendered nouns in order to flag obvious gendering but not debias it as in (Bolukbasi et al. 2016). There is no debiasing or normalizing here as all features have random initialization and are learned features. While there are no corrections, more explicit features leave less space for hidden relationships.

Multiple position features explicitly represent the complex relationships of word order. There are four positioning dimensions present: learned absolute positions, relative positions, relative positions within sets, and parse tree position. The absolute position token moves from its own set of tokens into the main feature set. Then there are relative positions like (Shaw et al. 2018) and relative positions within sets of parts of speech such as $noun-2$, $noun-1$, $noun+1$, $noun+2$ to indicate the previous or subsequent nouns. In addition to their qualitative meanings, the dependency tags and their head tags mark relative positions within the parse tree.

Table 1. Token vs Token Vectors of Various Features

Token Vector Size	Features	Accuracy	Perplexity
2	Surface Form+Absolute Position	0.0981	13525.84
8	+POS+Number+Root+Suffixes	0.1176	14371.32
22	+B Tags+Lemma+Ent_Type+Morph+Sub-Suffixes	0.1306	8173.72
23	+Dependency	0.2175	920.67
26	+Head Dependency+	0.2280	689.66
32	+Gender+Coreference+Relative Positions	0.2564	679.79

The token vectors represent context words as the aggregate of their features as in (Bian et al. 2022). Within the token vectors there are dimensional limits: exclusive subsets, $NONE$ features, and a hard vector length. Masked words have only the $MASK$ feature and absolute position. Target words are restricted to the surface forms. All the features of these token vectors require 3 times the original set of tokens for surface form features. The number of tokens for root words and lemmas scale with the number of surface form tokens. There are less than 1000 tokens for all other features.

There is a 5 word minimum frequency threshold, below which words are replaced with UNK. All digits are converted to NUM. All embeddings for all features are initialized with Xavier initialization.

For token vectors, minimum thresholding eliminates features but not entire words. Features of entries are not removed but replaced with UNK, and since other features provide a partial view, these are comparable to other incomplete entries like numbers whose numeric values are replaced with NUM.

Processing:

The classifier is a classic transformer encoder with source code from (Lewis Tunstall and Wolf 2022), a straight forward implementation of (Vaswani et al. 2017). A 32 dimension input replaces the 1 dimension input with position. The fusion function is *sum* and occurs in the call to Embedding.

3 Experiments

There are 6 tests in Table 1, begining with the baseline and then adding features. These are averages of tests with different random seeds.

Adding the parser features improves accurracy by over 2 times and reduces perplexity by almost 20 times. The drops in perplexity loosely scale with the rises in accuracy. The greatest contributions come from the features for dependencies. Dependency heads like all other features have modest gains. The features appear in groups to show the progression of improvements. The independent features in the 8, 22, and 32 tests are put into groups arbitrarily. Aside from dependency features, no independent feature ablative testing could overcome the noise of random retests, so they are bundled for testing. The 8 and 22 sets show the incremental progress of small features as gains between sets, and that it can happen without dependencies. And the 32 set continues that progress after dependencies. The first 5 tests ran in 6 epochs, after which overfitting diminished their results. The last test could run for 24 epochs before an impact from overfitting.

Other classic measurements of improvement become moot or lose impact with token vectors. The felicity of sub word tokens does not increase when they combine in a token vector. With word cosine similarity, the overlap of composible token vectors overshadows their difference. Word relationships now carry features that appear on both sides. For a Word2Vec (Mikolov, Yih, et al. 2013) example of X_{apple} - $X_{apples} \approx Xcar$ - $Xcars$, the left side expression X_{apple} - X_{apples} could be represented by $[apple_{string}, apple_{lemma}, apple_{root}, noun, singular,$

$Suffix_{NONE}$] - [$apples_{string}$, $apple_{lemma}$, $apple_{root}$, $noun$, $plural$, $Suffix_s$]. There is a difference between surface forms, but more importantly a difference of $plural$ + $Suffix_s$ and $singular$ + $Suffix_{NONE}$, as well as a similarity of $apple_{lemma}$ + $apple_{root}$ + $noun$.

4 Conclusions

Additional inputs start a model on a greater foundation of semantic content and of syntactic form for composing representations of relationships. A transformer model based on the surface forms would eventually contain the implicit syntactic and semantic relationships as latent relationships from the dataset. Explicitly moving a large segment of this information into the input raises the floor of model performance and generalization as measured by masked word prediction accuracy and perplexity.

References

J. Bian, B. Gao, and T.-Y. Liu. 2022. "Knowledge-Powered Deep Learning for Word Embedding." In: *Machine Learning and Knowledge Discovery in Databases*. Springer-Verlag, Nancy France, 132–148. ISBN: 978-3-662-44847-2. doi:10.1007/978-3-662-44848-9_9.

T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama, and A. Kalai. 2016. *Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings*. (2016). https://arxiv.org/abs/1607.06520 arXiv: 1607.06520 (cs.CL).

S. Egea Gómez, E. McGill, and H. Saggion. Sept. 2021. "Syntax-aware Transformers for Neural Machine Translation: The Case of Text to Sign Gloss Translation." In: *Proceedings of the 14th Workshop on Building and Using Comparable Corpora (BUCC 2021)*. Ed. by R. Rapp, S. Sharoff, and P. Zweigenbaum. INCOMA Ltd., Online (Virtual Mode), (Sept. 2021), 18–27. https://aclanthology.org/2021.bucc-1.4/.

Francis Kučera and W. Nelson Francis. 1979. *Brown Corpus Manual*. Providence, Rhode Island. http://icame.uib.no/brown/bcm.html.

M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd. 2020. *spaCy: Industrial-strength Natural Language Processing in Python*. (2020). doi:10.5281/zenodo.1212303.

H. Jabbar. 2024. *MorphPiece : A Linguistic Tokenizer for Large Language Models*. (2024). https://arxiv.org/abs/2307.07262 arXiv: 2307.07262 (cs.CL).

K. Lang. 1995. *20 Newsgroups Dataset*. Accessed: 2026-03-11. (1995). http://people.csail.mit.edu/jrennie/20Newsgroups/.

L. v. W. Lewis Tunstall and T. Wolf. 2022. *Natural Language Processing with Transformers*. O'Reilly. ISBN: 978-1-098-13679-6.

Y. Liu and Y. Hou. 2024. *Syntax-Aware Complex-Valued Neural Machine Translation*. (2024). https://arxiv.org/abs/2307.08586 arXiv: 2307.08586 (cs.CL).

M.-C. de Marneffe and C. D. Manning. 2008. "The Stanford Typed Dependencies Representation." In: *Proceedings of the Workshop on Cross-Framework and Cross-Domain Parser Evaluation*. Manchester, UK, 1–8. https://aclanthology.org/W08-1301.pdf.

T. Mikolov, K. Chen, G. Corrado, and J. Dean. Jan. 2013. "Efficient Estimation of Word Representations in Vector Space." *Proceedings of Workshop at ICLR*, 2013, (Jan. 2013).

T. Mikolov, W.-t. Yih, and G. Zweig. June 2013. "Linguistic Regularities in Continuous Space Word Representations." In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by L. Vanderwende, H. Daumé III, and K. Kirchhoff. Association for Computational Linguistics, Atlanta, Georgia, (June 2013), 746–751. https://aclanthology.org/N13-1090/.

T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. 2013. "Distributed Representations of Words and Phrases and their Compositionality." *CoRR*, abs/1310.4546. http://arxiv.org/abs/1310.4546 arXiv: 1310.4546.

P. Shaw, J. Uszkoreit, and A. Vaswani. 2018. *Self-Attention with Relative Position Representations*. (2018). https://arxiv.org/abs/1803.02155 arXiv: 1803.02155 (cs.CL).

N. Stylianou and I. Vlahavas. Dec. 2020. "E.T.: Entity-Transformers. Coreference augmented Neural Language Model for richer mention representations via Entity-Transformer blocks." In: *Proceedings of the Third Workshop on Computational Models of Reference, Anaphora and Coreference*. Ed. by M. Ogrodniczuk, V. Ng, Y. Grishina, and S. Pradhan. Association for Computational Linguistics, Barcelona, Spain (online), (Dec. 2020), 1–10. https://aclanthology.org/2020.crac-1.1/.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. "Attention Is All You Need." *CoRR*, abs/1706.03762. http://arxiv.org/abs/1706.03762 arXiv: 1706.03762.

X. Zhang, J. Zhao, and Y. LeCun. 2015. "Character-level Convolutional Networks for Text Classification." In: *Advances in Neural Information Processing Systems 28*, 649–657. http://www.di.unipi.it/~gulli/AG_corpus_of_news_articles.html.

A Online Resources

https://github.com/maconnor34/parse2vec

B Reproducibility Checklist for JAIR

Select the answers that apply to your research – one per item.

All articles:

(1) All claims investigated in this work are clearly stated. yes
(2) Clear explanations are given how the work reported substantiates the claims. yes
(3) Limitations or technical assumptions are stated clearly and explicitly. yes
(4) Conceptual outlines and/or pseudo-code descriptions of the AI methods introduced in this work are provided, and important implementation details are discussed. yes
(5) Motivation is provided for all design choices, including algorithms, implementation choices, parameters, data sets and experimental protocols beyond metrics. yes

Articles containing theoretical contributions:

Does this paper make theoretical contributions? no

If yes, please complete the list below.

(1) All assumptions and restrictions are stated clearly and formally. [yes/partially/no]
(2) All novel claims are stated formally (e.g., in theorem statements). [yes/partially/no]
(3) Proofs of all non-trivial claims are provided in sufficient detail to permit verification by readers with a reasonable degree of expertise (e.g., that expected from a PhD candidate in the same area of AI). [yes/partially/no]
(4) Complex formalism, such as definitions or proofs, is motivated and explained clearly. [yes/partially/no]
(5) The use of mathematical notation and formalism serves the purpose of enhancing clarity and precision; gratuitous use of mathematical formalism (i.e., use that does not enhance clarity or precision) is avoided. [yes/partially/no]
(6) Appropriate citations are given for all non-trivial theoretical tools and techniques. [yes/partially/no]

Articles reporting on computational experiments:

Does this paper include computational experiments? yes

If yes, please complete the list below.

(1) All source code required for conducting experiments is included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for source code readability and documentation as well as for long-term accessibility. NA
(2) The source code comes with a license that allows free usage for reproducibility purposes. NA
(3) The source code comes with a license that allows free usage for research purposes in general. NA
(4) Raw, unaggregated data from all experiments is included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for long-term accessibility. yes
(5) The unaggregated data comes with a license that allows free usage for reproducibility purposes. yes
(6) The unaggregated data comes with a license that allows free usage for research purposes in general. yes
(7) If an algorithm depends on randomness, then the method used for generating random numbers and for setting seeds is described in a way sufficient to allow replication of results. NA
(8) The execution environment for experiments, the computing infrastructure (hardware and software) used for running them, is described, including GPU/CPU makes and models; amount of memory (cache and RAM); make and version of operating system; names and versions of relevant software libraries and frameworks. yes

(9) The evaluation metrics used in experiments are clearly explained and their choice is explicitly motivated. yes
(10) The number of algorithm runs used to compute each result is reported. yes
(11) Reported results have not been "cherry-picked" by silently ignoring unsuccessful or unsatisfactory experiments. yes
(12) Analysis of results goes beyond single-dimensional summaries of performance (e.g., average, median) to include measures of variation, confidence, or other distributional information. no
(13) All (hyper-) parameter settings for the algorithms/methods used in experiments have been reported, along with the rationale or method for determining them. yes
(14) The number and range of (hyper-) parameter settings explored prior to conducting final experiments have been indicated, along with the effort spent on (hyper-) parameter optimisation. NA
(15) Appropriately chosen statistical hypothesis tests are used to establish statistical significance in the presence of noise effects. NA

Articles using data sets:

Does this work rely on one or more data sets (possibly obtained from a benchmark generator or similar software artifact)? yes

If yes, please complete the list below.

(1) All newly introduced data sets are included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for long-term accessibility with a license that allows free usage for research purposes. yes
(2) The newly introduced data set comes with a license that allows free usage for reproducibility purposes. yes
(3) The newly introduced data set comes with a license that allows free usage for research purposes in general. yes
(4) All data sets drawn from the literature or other public sources (potentially including authors' own previously published work) are accompanied by appropriate citations. yes
(5) All data sets drawn from the existing literature (potentially including authors' own previously published work) are publicly available. yes
(6) All new data sets and data sets that are not publicly available are described in detail, including relevant statistics, the data collection process and annotation process if relevant. NA
(7) All methods used for preprocessing, augmenting, batching or splitting data sets (e.g., in the context of hold-out or cross-validation) are described in detail. NA

Explanations on any of the answers above (optional):

The results are single-dimensional on purpose. This paper shows only one change that is easy to implement.